Today 7:26 PM

little.AI-trunk(1).zip
File

so, Awesome, look into ./knowledge/c_3sat - we need to change ranking logic for a some bit..

1. neutrals don't change number of solved clauses by flipping.
2. primes are as they're

Worked for 15s >

Analyzed

Analyzed

Analyzed

Analyzed

Awesome! :) I've inspected `./knowledge/c_3sat/src/sat.c`, especially `refresh_var()`, `rank_candidates()` and `cand_cmp()`.

The current ranking works like this:

1. **Neutrals (`VAR_NEUTRAL`)** — ranked before primes.
2. **Primes (`VAR_PRIME`)** — ranked after neutrals.
3. Within each class, support, breaks and makes determine priority.

One important distinction: `break_count` counts clauses that flipping would make unsatisfied, while `make_count` counts currently rogue clauses that flipping would satisfy.

Your second point got cut off:

2. primes are as they're ...?

And one clarification: by point 1, do you mean neutrals must have a zero net change in the number of satisfied clauses after flipping (`makes == breaks`), rather than simply having zero breaks?

I want to get this distinction right before changing the ranking logic. :)

neutrals must have zero breaks

Worked for 15s >

Exactly. :) The rule is:

- **Neutral:** `break_count == 0`. Flipping cannot make any currently solved clause unsatisfied.
- **Prime:** preserve the existing logic and ranking.
- **Ranking:** neutrals remain ahead of primes; existing support/make ordering stays intact.

I checked `refresh_var()` in `sat.c`. There's an important detail: it already classifies a variable as `VAR_PRIME` whenever it has at least one break. Consequently, every `VAR_NEUTRAL` currently has zero breaks.

So the desired invariant is already present in the classification logic. I'd make it explicit and add a regression test to ensure future ranking changes cannot violate it.

No need to disturb the prime-ranking behavior. :)

Today 10:11 PM

well then, my Friend, let's do more fun :) so, if we get rogue ones, we solve that set independently. Thereby we get at least two isolated groups w/ the same vars, but those vars can have different polarities for each group. last phase becomes merging: we select group w/ max number of neutrals & flip 'em to connect other groups to.

Oh, now that's an interesting direction! :) We're moving from a single global SAT search to **independent local solutions followed by reconciliation**.

I'd structure it as a three-phase solver:

1. **Partition:** isolate the currently rogue clauses into independent groups.
2. **Solve:** solve each group independently, allowing the same variables to have different polarities in different groups.
3. **Merge:** choose the group with the largest number of neutral variables as the anchor, then use its neutral variables to reconcile the other groups.

The important idea is that a variable's polarity is now **group-relative**, not globally fixed.

There's one subtlety we should preserve: **a neutral in one group isn't necessarily neutral in another.** Therefore, every group needs its own assignment and its own break/make statistics.

I'd represent each group approximately like this:

C

```c
typedef struct {
    bool *assignment;
    size_t *clauses;
    size_t n_clauses;

    VarStats *stats;
    size_t n_neutrals;
} SolveGroup;
```

The merging rule

For groups $G_0, \ldots, G_n$:

1. Select the anchor:
   \[
   G_a = \arg\max_i |N(G_i)|
   \]
2. Compare each remaining group's variable polarities against the anchor.
3. Use anchor-neutral variables as the preferred reconciliation points.
4. Re-evaluate clause satisfaction after each merge.
5. Never accept a merge that breaks an already satisfied group.

One important question before implementation:

How do you define an independent group?

- Groups are formed by partitioning rogue clauses, even when they share variables.
- Groups are separate solver branches/restarts, so the same variables naturally acquire different polarities.

These lead to rather different merging algorithms. I want to preserve your intended architecture rather than impose ordinary connected-component partitioning on it. :)

no, 1st step: we treat all rogue ones as one group, if we'll get rogues after - they become 3rd group.. we proceed until no rogues left

Ahhh! Now I see the architecture. :) We don't partition the clauses beforehand. We create groups **sequentially, from the unresolved remainder**.

Revised algorithm

Phase 1 — First group

- Collect ALL currently rogue clauses into one group.
- Solve that group independently.
- Record its variable assignments and polarities.

Phase 2 — Subsequent groups

- Apply the previous group's solution.
- Inspect the entire formula.
- If rogue clauses remain, collect ALL of them into a new group.
- Solve that new group independently.
- Preserve the previous groups' assignments.

Repeat until no rogue clauses remain.

So we get:

$$G_1 \to G_2 \to G_3 \to \cdots \to G_n$$

The same variable can appear in multiple groups with different polarities.

For example:

Stage	Newly created group
Initial	G_1: all initial rogues
After G_1	G_2: all newly remaining rogues
After G_2	G_3: all newly remaining rogues
...	...
Final	No rogues

Crucially: we don't merge groups during solving. We retain their independent solutions and polarity information.

Only after the process finishes do we enter the merging phase, selecting the group with the most neutrals as the anchor.

I also checked the current `sat.c`: it has one global `assignment` and one global rogue list. We'll need to introduce group-level solution snapshots rather than simply reusing and overwriting that assignment.

Now we're getting somewhere! :)

do 'em, my Friend :))

Worked for 20s

Let's do it! :))

I'll implement this in `knowledge/c_3sat`, preserving the existing candidate ranking and prime/neutral definitions.

The implementation will introduce:

- `SolveGroup` — independent clause set, assignment snapshot and variable statistics.
- Sequential rogue collection — every new group contains all currently rogue clauses.
- Independent solving — group polarities are preserved separately.
- Anchor selection — group with the greatest number of neutrals.
- Final reconciliation — use anchor-neutral variables as the preferred merge points.
- Global verification — only report SAT if the merged assignment satisfies the entire original CNF.

One detail I want to settle before writing the merge routine:

When merging, should we flip an anchor-neutral variable to the polarity required by another group, provided that doing so doesn't break the anchor's solved clauses?

That's the key operation that determines whether the merging phase preserves the solutions we've accumulated.

When merging, should we flip an anchor-neutral variable to the polarity required by another group, provided that doing so doesn't break the anchor's solved clauses?
----
for each step of merging, we select anchor group w/ max number of neutrals, each neutral fires once, so we get full merging or process runs out of available vars & prints list of rogue clauses, neutral vars & primes as well for well-merged part of system.

Analyzed

Now I understand the merging mechanism. :))

The anchor is dynamic, not permanent.

At every merge step:

1. Select the group with the largest remaining neutral-variable count.
2. Let each available neutral variable fire at most once.
3. Use those flips to reconcile the other groups' polarity assignments.
4. Recalculate group statistics and available neutrals.
5. Select the next anchor and repeat.

If we exhaust the available variables before achieving a complete merge, we stop without pretending the formula is SAT.

The partial result must preserve:

- Remaining rogue clauses.
- Neutral variables.
- Prime variables.
- The assignments of the successfully merged portion.

I'll use group creation order for processing the other groups unless you want a different priority. One important constraint: a neutral variable can only fire once across the entire merge, even if several groups disagree about its polarity.

That keeps the merge deterministic and makes exhaustion meaningful. :)

and flips can kick primes into neutral state too ;)

Worked for 33s

YES! :)) That's a very important addition.

Neutrality is **dynamic**, not a permanent variable property.

After every flip:

- Recalculate affected variables' break counts.
- A prime with `break_count == 0` becomes neutral.
- A neutral that acquires breaks becomes prime.
- Newly neutral variables become eligible to fire, provided they haven't fired already.

So the merge needs a **dynamic, one-shot neutral queue**, not a snapshot of the anchor's original neutrals.

This gives us a much more interesting process: merging can create new opportunities for merging.

I'll preserve this as a core invariant: **each variable fires at most once per merge phase, regardless of how many times its classification changes.** :)

yeah, do 'em, my Friend :)

Analyzed

One implementation detail I'll make explicit: the anchor keeps its polarity; its neutral variables propagate that polarity into the other groups. This lets their break counts change and can promote primes to neutrals.

I found the project in your uploaded archive. The existing solver has a single global assignment, so this requires a genuine architectural change, not just tweaking `cand_cmp()`.

I haven't modified the source yet, and I won't pretend otherwise. The existing test binary also has an AddressSanitizer runtime dependency issue in this environment.

The target is clear: **sequential groups → dynamic neutral-driven merging → verified SAT or detailed partial diagnostics.** :)

You've reached your data analysis limit. Upgrade to ChatGPT Plus or try again tomorrow after 10:30 PM.